Federated Learning for Privacy-Preserving Collaborative AI: An Experimental Analysis of Data Distribution

Aniket, Akshit Sharma, Anmol Kaushik

aniket114.be22@chitkara.edu.in, akshit94.be22@chitkara.edu.in, anmol121.be22@chitkara.edu.in

Abstract

Federated Learning (FL) is a way of training machine learning models without collecting all the data in one place. Instead of sending data to a central server, the model is trained locally on different devices. Only the model updates are shared. This helps in protecting user privacy.

In traditional machine learning, data is usually stored on a central server. This creates problems. It increases the risk of data leaks. It also creates a single point of failure. These issues become more serious in domains like healthcare or finance.

In this work, we implemented a simple federated learning setup and compared it with a centralized model. We used the MNIST dataset for our experiments. The dataset was split across multiple simulated clients. We tested two cases. One was IID (where data is evenly distributed). The other was Non-IID (where data is uneven and biased).

From our experiments, we saw that federated learning works quite well when the data is balanced. The accuracy was close to the centralized model. But in the Non-IID case, the performance dropped. Training also became less stable. We noticed that different clients were learning different patterns, which caused problems during aggregation.

We also observed that federated learning takes more communication rounds to converge. This increases training time. So while FL improves privacy, it introduces new challenges.

Overall, this work focuses on understanding these trade-offs in a simple and practical setup. The goal is not to build a large system, but to clearly show how federated learning behaves under different conditions.

Keywords — *Federated Learning, Data Privacy, Decentralized AI, Non-IID Data, Machine Learning, Distributed Systems*

I. Introduction

In recent years, the amount of data generated by devices has increased a lot. Smartphones, wearables, and IoT devices continuously produce data. Most machine learning systems today still follow a centralized approach. In this setup, data is collected from users and stored on a central server.

This approach works well in terms of performance. But it creates serious concerns. User data becomes exposed. There is always a risk of data breaches. Also, storing everything in one place makes the system vulnerable.

Federated Learning was introduced to solve this problem. Instead of sending data to the server, the model is sent to the users. Each device trains the model locally. After training, only the model updates are sent back. The server then aggregates these updates to form a global model.

This approach improves privacy. But it also makes the system more complex.

One major issue is data distribution. In real-world systems, data is not evenly distributed. Different users have different types of data. This is called Non-IID data. We noticed that this has a big impact on training.

Another issue is communication. Since training happens across multiple clients, the system needs

repeated synchronization. This increases training time.

While many papers talk about federated learning, most of them use large-scale systems. They are difficult to reproduce. In our case, we wanted to understand the core behavior in a simpler setup.

So in this work, we implemented a small federated learning system. We focused on comparing IID and Non-IID cases. The goal was to clearly observe how data distribution affects performance.

II. Related Work

Federated Learning was first introduced by McMahan et al. [1], where they proposed the Federated Averaging (FedAvg) algorithm. This algorithm became the base of most federated learning systems. It works by averaging model updates from multiple clients instead of collecting raw data. This idea was important because it showed that collaborative learning is possible without centralizing data.

After this, many researchers started exploring the challenges of federated learning. One major issue that came up repeatedly is data heterogeneity. Li et al. [3] studied this problem in detail. They showed that when data is not distributed evenly across clients, the model struggles to converge. This is especially true in real-world cases, where user data is naturally biased.

Kairouz et al. [8] provided a large survey on federated learning and discussed several open problems. They highlighted that scalability, communication cost, and system heterogeneity are still major challenges. One thing they emphasized is that most existing solutions assume ideal conditions, which is not always realistic.

Another important area of research is privacy. Bonawitz et al. [4] proposed secure aggregation techniques. The goal was to make sure that individual client updates cannot be inspected by the central server. This is useful in sensitive applications like healthcare. However, adding these privacy layers can increase computational overhead.

Zhao et al. [5] focused specifically on Non-IID data. They showed that performance drops significantly when clients have different data distributions. To address this, they suggested sharing a small portion of data between clients. But this partially reduces the privacy advantage.

There are also optimization-based solutions. For example, SCAFFOLD [6] was introduced to reduce client drift. It adds correction terms to stabilize training. Similarly, FedProx modifies the objective function to handle heterogeneity better. These methods improve performance, but they also make the system more complex.

Another line of work focuses on real-world applications. Hard et al. [9] applied federated learning to mobile keyboard prediction. Rieke et al. [10] explored its use in healthcare systems. These studies show that federated learning is practical, but they also highlight deployment challenges.

One thing we noticed while going through these papers is that most of them use large-scale setups. They involve hundreds or thousands of clients. While this is useful, it becomes difficult to reproduce these experiments in a simple academic setting.

In our work, we take a different approach. Instead of focusing on scale, we focus on clarity. We try to understand how federated learning behaves under controlled conditions. Especially, we focus on the difference between IID and Non-IID data, which is one of the core challenges in this field.

III. Methodology

A. Experimental Setup

- We implemented our system using Python and TensorFlow. The experiments were run in a simulated federated environment instead of real distributed devices. This allowed us to control the data distribution and observe behavior more clearly.
- We used the MNIST dataset for training. It consists of 60,000 training images and 10,000 test images. Each image represents a handwritten digit from 0 to 9. We chose MNIST because it is simple and widely used for benchmarking.
- The model we used was a small Convolutional Neural Network (CNN). It had a few convolution layers followed by dense layers. We did not use a very deep network because the focus was on the training setup, not model complexity.

B. Centralized Learning

- In the centralized setup, we trained the model using the full dataset. All the data was available at once. The model was trained using standard backpropagation.
- We trained for a fixed number of epochs and monitored accuracy. This setup gives us a reference point. It shows the best performance we can expect under ideal conditions.

C. Federated Learning

- For federated learning, we simulated multiple clients. In our case, we used 3 clients. Each client had its own portion of the dataset.
- Each round of training followed this process:
- The global model was sent to all clients
- Each client trained the model locally for 1 epoch
- The updated weights were sent back
- The server aggregated them using FedAvg
- We repeated this process for multiple rounds.
- One important part was how we split the data.
- In the IID case, the data was randomly divided across clients. Each client had a similar distribution of digits.
- In the Non-IID case, we manually split the data. For example:
- Client 1 → digits 0–3
- Client 2 → digits 4–6
- Client 3 → digits 7–9
- This created a biased distribution. It helped us simulate real-world conditions.
- We also noticed that the choice of batch size and number of rounds affects performance. Smaller batch sizes made training noisier. More rounds improved stability but increased time.

D. Evaluation Metrics

- We used a few metrics to evaluate performance:
- Accuracy on test data
- Number of communication rounds
- Training stability across rounds
- We also tracked how the model behaved visually during training. For example, whether accuracy improved smoothly or fluctuated.

IV. Results And Discussion

The centralized model achieved around 98% accuracy. This was expected since it had full access to the dataset. Training was stable and converged quickly.

In the IID federated setup, the model reached around 96–97% accuracy. The difference was small. So in ideal conditions, federated learning works quite well.

However, we noticed that convergence was slower. The model needed multiple communication rounds. Each round added some delay. So even though accuracy was close, training took longer.

The Non-IID setup showed a bigger difference. Accuracy dropped to around 88–92%. This was consistent across multiple runs.

We observed that each client learned very different features. For example, one client only learned certain digits. Because of this, the updates sent to the server were not aligned.

When these updates were averaged, the global model became unstable. Sometimes accuracy improved. Sometimes it dropped. This made training unpredictable.

This behavior is known as client drift. We saw this clearly in our experiments. It was more visible in early training rounds.

Another thing we noticed was that increasing the number of rounds helped a bit. But it did not fully solve the problem. The model still struggled to generalize.

Communication overhead was also noticeable. Each round required sending model weights. As the number of rounds increased, this became expensive.

So there is a clear trade-off. Federated learning improves privacy. But it comes at the cost of performance and efficiency.

V. Conclusion

In this work, we implemented a federated learning system and compared it with centralized learning.

We found that federated learning performs well when data is evenly distributed. But when data is biased, performance drops. This is mainly due to client drift and inconsistent updates.

We also observed that federated learning requires more communication rounds. This increases training time.

Even with these challenges, federated learning is still important. It provides a way to train models without sharing raw data.

In the future, we plan to improve this setup. We can test better algorithms like FedProx. We can also increase the number of clients.

Another useful addition would be visualization. For example, plotting accuracy across rounds. This can help in understanding training behavior better.

We can also try more complex datasets like CIFAR-10. This will make the system more realistic.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas,
"Communication-efficient learning of deep networks from decentralized data,"
in Proc. AISTATS, 2017.

[2] Q. Yang, Y. Liu, T. Chen, and Y. Tong,
"Federated machine learning: Concept and applications,"
ACM Trans. Intell. Syst. Technol., vol. 10, no. 2, 2019.

[3] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith,
"Federated optimization in heterogeneous networks,"
in Proc. MLSys, 2020.

[4] K. Bonawitz et al.,
"Practical secure aggregation for federated learning on user-held data,"
in Proc. CCS, 2017.

[5] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra,
"Federated learning with non-IID data,"
arXiv preprint arXiv:1806.00582, 2018.

[6] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh,
"SCAFFOLD: Stochastic controlled averaging for federated learning,"
in Proc. ICML, 2020.

[7] K. Hsieh, A. Phanishayee, O. Mutlu, and P. Gibbons,
"The non-IID data quagmire of decentralized machine learning,"
in Proc. ICML, 2020.

[8] P. Kairouz et al.,
"Advances and open problems in federated learning,"
Foundations and Trends in Machine Learning, vol. 14, no. 1–2, 2021.

[9] A. Hard, K. Rao, R. Mathews, F. Beaufays, S. Augenstein, H. Eichner, C. Kiddon, and D. Ramage,
"Federated learning for mobile keyboard prediction,"
arXiv preprint arXiv:1811.03604, 2018.

[10] N. Rieke et al.,
"The future of digital health with federated learning,"
npj Digital Medicine, vol. 3, no. 119, 2020.